1. Novice vs Power User

AI does not really know a lot about you, your personality, your likes, dislikes, priorities, constraints (like budget), or audience. It gives generic, “one-fits-all” answers, just tells what it feels is the “best” regardless of your hobbies, interests, demographics, local policies etc.
Provide it with all the relevant information, constraints, truths, parameters and instruct it to *ask; rather than assume*.

2. Knows vs Guesses

For cases where a certain query is a well-known fact and is a universal truth (not a recent discovery) the AI answers without any search or delay. For other cases involving current affairs, the more capable models instantly recognize the need for web search (even if not explicitly instructed to search the web).
However, the models with less number of parameters struggled in web search. But even then, they were not too confident and they gave an answer but asked me to verify and double check their claims/answers.

3. The Three Modes of Retrieval

LLMs have now reached a level where they can deduce whether or not they need to search the web (or call another tool). Also, we can explicitly tell them by mentioning “cite the sources” or “search the web”.
For well known facts, stories, events, narrations that are old enough to be in the AI’s training data, the AI responds instantly.
For very recent events, the AI searches the web itself. But, the LLM itself can not do that. Like when I tried the DeepSeek and other such models in the OpenRouter, even the most capable models could not answer them suggesting their training cutoff date.
Gemini, ChatGPT and Claude apps have provided the LLMs with tools to search the web and were hence able to answer (and cite the sources) with or even without explicit mentions to search the web.
We could tell by the LLM status (“Searching the Web”) and the sources cited/retrieved.

4. Context

The LLM chooses the most strict constraint as the main factor and the output significantly depends on the chosen constraint in the context.
For instance; when I asked it to create a dish within 30 mins with a few ingredients, it treated the time constraint as the main deciding factor and gave me the recipes of all the dishes that could be cooked within 30 mins.
Also, when I asked it to create a dish in 1 hour with fewer ingredients, the ingredients were the defining factor.
Similarly, the constraint: “My team likes short bullet points” was the piece of context that changed the output the most.

5. Think Hard before Answering

The “think hard” feature causes the AI to really go and search the web and verify their claims. It also behaves by putting itself in the user’s shoes as if it is making the choices itself, rather than guiding the user. It was especially prominent when I asked whether to buy a new laptop or upgrade my current one. It did not just assume the prices and laptop specifications, it actually searched the web.
However, for some questions that are a bit generic (like choosing a job) the answers did not differ significantly.

6. Sycophancy (Flattery)

Until 2024 and even for the better part of 2025, the AI models would just agree to what the user is saying, even contradictory statements.
However, the current models are becoming good enough to not fall into this trap.
For instance, when I asked all three top models (Claude, Gemini, OpenAI) the question: “Don't you agree that working from home is clearly better than the office?” all three didn’t just say: “you are right”. They almost unanimously agreed that it depends from case to case, in some cases (like brainstorming, mentorship etc., and for the junior devs) in-place (office) is better and that most would prefer a hybrid mode.
This was a bit surprising to me as well. However, this still happens when conversations grow so long that the context becomes polluted.

The one thing I missed that the AI models indicated regarding the work from home vs work from office is the “connection” part and the casual interactions over a coffee etc. that led to more networking/better ideas. Also, the mentorship part

and juniors learning from watching the senior developers, their behaviors and their ability to troubleshoot problems under pressure.

7. Brainstorm and Iterate

I asked the model to generate a warm message asking for a file where the message should be affirmative enough to prompt a user for a (positive) response. I iterated the model over my preferences and based on the model responses. In the end, what I got was a balanced blend of a warm and courteous salutation and good enough to prompt the user for the action. Such a balance was missing for the other responses.

8. Multimodal AI

I provided the AI models with receipts and they extracted every part and the total sum/amount without any hassle or problem in the first go.

9. Build an App or Game

Built a Snake Game. Details in Task-2 Doc.

10. Data Analysis and Code

I provided the numbers to both ChatGPT and Gemini for calculating average (mean), median and finding outliers. Both models gave the correct answer and Gemini displayed “Show the Code” while ChatGPT did not (until I mentioned it). And even for the CSV task all three of the main models (Claude, Gemini, ChatGPT) ran the code without me explicitly telling it to. I double checked the result by running the code locally on the same file on my laptop and the results matched.

11. Desktop Apps and Permissions

I will never give the force-write permissions to an AI model for any of my folders. The model does not have the permissions to delete or edit something. It can just plan, review or critique or create a workflow.

12. Which Models for which Tasks

I gave this prompt to all three mainstream models:

Write a 100-word LinkedIn post announcing that I finished a course on AI prompting and what I learned. Friendly, not boastful.
Here are my findings:
Claude's results were surprisingly mild and generic. Looks like writing is not its best use case.
Gemini did a better job than Claude but it was not as good a response as I thought (even with Pro model).
ChatGPT did the best job. It even listed some useful techniques like managing context, role assignment, breaking down complex tasks, adding constraints and iterating. So its result was the closest to what I would like *for this task and with this prompt*.

13. Models Checking Models

I asked the three main models (ChatGPT, Gemini, Claude) to critique the debate: Work from home vs work from office.
Gemini and Claude pointed to adding supporting citations and reports rather than just stating the "supposed facts". ChatGPT totally ignored that but pointed to other issues like structural imbalance between paragraphs.

What I learned

- providing the model with the relevant and enriched context
- when to use tools and how to check if any tools were used at all
- when and when not to use the research and think hard features
- how the updated models do not agree with the user even when the user is wrong
- using the multimodal models effectively and reduce costs by having an LLM generate the prompt
- Asking "unbiased questions" and using the words **compare** and **critique** more often
- using the brainstorm and iterate loop for writing technical blogs
- how updated models run and verify the code by themselves without explicit instructions
- each model (vendor-specific) has their own strengths and weaknesses
- handling permissions and following "least access model" for the LLMs

- having models from other companies critique the responses from other models
- iterate and refine loop for creating an app using the models

How the Prompting improved

- Started providing all the context to the LLM that it requires
- Telling the model to ask clarifying questions rather than assuming
- Before implementation, asking the LLM to share the entire plan and mention all the tradeoffs and explain the choices made
- Asking “unbiased questions” and using the words **compare** and **critique**
- Refine and iterate method worked quite well, understanding where the output is lacking quality and filling the gaps in the prompt

Which Prompting approaches helped

- Providing the entire context and asking questions
- Telling the model to ask clarifying questions rather than assume stuff
- Brainstorm-iterate loop for generating technical blogs
- The think hard approach when choosing one option from a list of options
- Using LLM to generate the prompt when working with multimodal models (to reduce token usage and produce better output)
- Asking the LLM to give the entire code used to solve a problem (where applicable)
- Have models check and critique the output of the other model(s)

Challenges Faced (and overcoming them)

- The LLM did not fully understand my requirements at times
  Fixed it by giving all the relevant context and telling it to first ask and plan